Security-Centric Software Development: Scenario-Based Analysis of Secure Coding Practices Across the SDLC

Lana N.R. Alnajjar
School of Engineering, Applied Science, and Technology
Canadian University Dubai
Dubai, United Arab Emirates
20220002177@students.cud.ac.ae

Nada S. Ibrahim
School of Engineering, Applied Science, and Technology
Canadian University Dubai
Dubai, United Arab Emirates
20220001333@students.cud.ac.ae

Ibrahim Adel
School of Engineering, Applied Science, and Technology
Canadian University Dubai
Dubai, United Arab Emirates
20220001397@students.cud.ac.ae

***Abstract*— The expanding reliance of today's institutions on software intensive systems has turned security into an essential quality rather than a feature add-on. Yet, on many projects security is still addressed reactively, at the latter stages of the software development lifecycle (SDLC), exposing portfolios to risk and requiring expensive rework that increases an organization's attack surface. Inspired by Khair Abdul's work on security- oriented software development, which recommends incorporation of secure coding (desirable dataset identification), this paper explains the process of achieving such an approach in practical Software Engineering contexts. The work takes a secondary approach, using literature to conceptually relate the secure coding guidelines and activities found in their research to the standard activities that are present in SDLC phases. It also characterizes the organizational and technical obstacles to adoption and discusses their implications for software security, quality and maintainability. The result is a well-defined problem statement and context for scenario-based analysis to help practitioners (and students in engineering security into software, rather than bolting it on later.**

Keywords—Security-centric software development, secure coding, software development lifecycle (SDLC), secure software engineering, software security, DevSecOps

I. Introduction And Problem Definition

A. Introduction

In today's hyper-connected digital world, software systems are the backbone of mission critical business transactions and activities as well as end-user engagements and citizen services. Meanwhile cyberattacks are increasing in frequency and sophistication, exploiting vulnerabilities in insecure or poorly engineered software. Security breaches not only damage the confidentiality, integrity and availability of data, but also bring huge financial loss, legal liability and brand erosion for the enterprises.

The classic approaches of the software development life cycle are traditionally not concerned about security, and more on functionality, performance and time-to-market. It is rarely a forethought in development, failing to be addressed until much later in the process: systems testing or release time. Such a reactive stance to security only serves to make vulnerabilities harder and more expensive to fix, and leaves systems open to known classes of attacks.

To mitigate these shortcomings, Khair proposes a security- centric SDLC process in which security is integrated at each stage of the SDLC and adopts systematic secure coding practices with supporting activities including risk analysis, code review, and testing [1]. Rather than treating security as a bolt-on activity or some final "hardening" step, they build software with security as a first-class consideration: the features designed for security, implemented security mechanisms and interfaces are tested for vulnerabilities through various techniques (e.g. black-box penetration testing), and so on.

Based on the. CYS-411 Engineering Secure Software course project, this paper uses Khair's security-oriented vision of an SDLC as a main reference and attempts to convert its ideas into practical, case-assisted advice. The aim is not to create or use a given system, but rather to help address the concepts of secure coding and activities insecurity within representative software projects throughout their lifecycles.

B. Problem Definition

Although secure coding guidelines, standards, and security frameworks are available, it is challenging for many organizations to make effective use of them throughout the software development lifecycle. Security needs are often insufficiently specified; developers may not be experts in secure coding; and security work is sometimes seen as an expensive overhead to be scheduled against deliveries. Therefore, there is still a considerable gap between high-level recommendations and day-to-day engineering decisions in actual software projects.

This project addresses the following central problem: *How can secure coding practices be systematically integrated into each phase of the SDLC in typical software development settings, and what challenges and trade-offs arise when doing so?*

More specifically, the work focuses on:

1) Synthesizing the key secure coding practices and security activities proposed in the literature, with emphasis on the security-centric SDLC approach described by Khair [1].
2) Conceptually mapping these practices to realistic software development scenarios and SDLC phases

(requirements, design, implementation, testing, deployment, and maintenance).

3) Identifying organizational, technical, and process-related barriers that hinder adoption of security-centric software development and outline their impact on software security and quality.

By framing the problem in this way, the paper provides a foundation for later sections, which will apply the synthesized practices to concrete scenarios and critically evaluate their effectiveness and limitations.

II. Background

A. Security-Centric Software Development and the SDLC

There is a typical flow of software development lifecycle (SDLC) that includes requirements, design and architecture, coding/implementation, testing & QA, deployment, and maintenance. In most legacy models, serious security work is limited to testing (such as penetration testing or even Day 2 operational control checks after the workload has been launched).

Security-oriented software development models apply to displacing these mind-sets towards viewing security as a component of all stages of the SDLC and not a distinct issue. Khair defines this as integrating lifecycle of secure coding practices, risk assessment and security-validation in such a way that, security requirements are specified early, carried into design and implemented into code as well as tested and monitored continuously [1]. This secure-by-design approach aligns with contemporary DevSecOps, which emphasizes automation and development and implementation pipelines that incorporate security.

Under this view, the SDLC is not just a functional delivery process, but a formal construct, a systematic approach to addressing security risks so that the security requirements are addressed systematically throughout the software life cycle (between inception and retirement).

Recent systematic literature review work confirms that integrating security practices into each phase of the SDLC remains a challenge for many organizations [4].

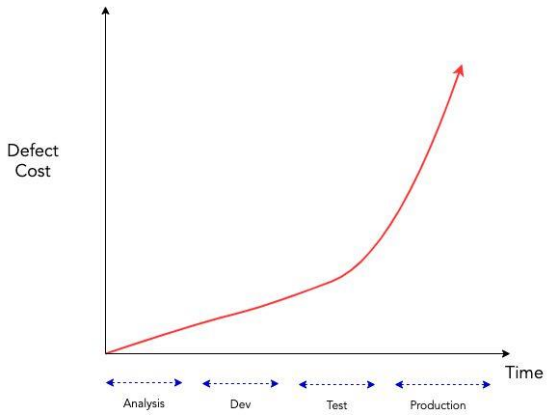

Fig. 1 – Boehm's Law

B. Secure Coding Practices

Secure coding practices refer to actual guidelines, patterns and techniques that developers use to minimize the chances of injecting vulnerabilities to software. Considering the synthesis suggested by [1], the following types of secure coding practice should be specifically relevant:

- **Principle of Least Privilege:** User, component and ser- vices must have access rights permitting them to do only what is necessary when carrying out their duties. Minimizing code and configuration privilege can minimize the effects of a compromise and minimize the lateral movement inside the system.
- **Input Validation and Sanitization:** Pertinent input must be validated against stringent conditions (type, length, format, range) and must be sanitized. Strong validation and sanitization are useful in avoiding the injection attack like SQL injection and cross-scripting.
- **Output Encoding:** Output encoding Making data that is displayed on user interfaces or within HTML or JavaScript code must be encoded correctly to make user-provided content is not seen as executable code. One of the ways to protect against cross-site scripting is a correct output encoding.
- **Authentication and Authorization:** Authentication should be a system that is reliable in assuring user identity (such as the secure management of passwords and where needed, multifactor authentication), whereas authorization logic should implement a fine-grained access control system (depending on user roles and permissions). These two factors are important to ensure that sensitive resources are not accessed by unauthorized individuals.
- **Secure session management:** Secure session management Sessions must have strong unpredictable identifiers, secure transport implementation (such as HTTPS), proper anti-timeout mechanisms, and anti-fixation and anti-hijacker protection. The session state should be man- aged to avoid that the attackers will hijack legitimate sessions.
- **Secure error handling and logging:** There should be no sensitive internal information (stack traces, SQL queries, or file paths) written down as error messages, and sufficient information must be logged to be useful in the investigation of the incident without any secrets. This assists in avoiding the disclosure of information and with the forensic investigation.
- **Secure configuration management:** Configuration that is of security relevance to the system (encryption parameters and keys and credentials and access control rules and default settings) should be through secure baselines and by industry best practices. The common cause of

vulnerabilities in deployed systems is misconfigurations.

These practices are effective leavers that developers and architects can use in the implementation of features. They are not scattered pieces of advice in a security-focused SDLC, but are organized to linked to requirements, design choices, and testing processes.

C. Challenges, Benefits, and Prior Work

Khair's study uses a secondary data-based review method- ology, analyzing research papers, industry reports, and other scholarly sources to identify how secure coding practices are proposed and applied across the SDLC, as well as the obstacles that organizations face [1]. The review highlights several recurring challenges:

• **Knowledge and Training Deficiency:** Developers and decision makers may not have a good understanding of secure coding principles or the ramifications of security decisions made by default.

• **Resources and Cost Constraints:** Budget constraints, time pressures, competitive pressure and priorities can make the investment in security education, tools, or processes hard to justify.

• **Complexity of Integration:** Adding new security activities and sources into the existing development process is, technically, as well as also organizationally challenging especially in legacy environments.

• **Compliance and Regulatory Requirement:** Industry specific organizations need to adhere to several standards, regulations which shun the security engineering inefficient in security if it does not properly align with development process.

• **Lack of Uniformity:** There are several overlapping guidelines and frameworks leading to confusion and non-uniform adoption of secure coding practices [5].

At the same time, the literature lists significant and consistent benefits in case secure coding and security-centric practices are implemented: improved security posture, decreased exploitable vulnerabilities, increased reliability and maintain- ability of systems, increased user trust, and cost savings over time, in terms of the number of incident response and rework activities and/or their cost [1].

This background is the impetus for the scenario-based analysis conducted in subsequent parts of this project. By an-choring the scenarios within an existing security-centric SDLC structure and proven secure coding practices, the purpose of the paper is to help bridge the gap between theoretical recommendations to development practices in real-world software development scenarios.

III. Proposed Approach and Analysis

This section presents three detailed security-oriented scenarios and analyzes them using the conceptual framework introduced in Khair's Security-Centric Software Development paper. Khair's work provides two primary contributions: (A) an analytical explanation for why software systems remain insecure when security is applied reactively, and (B) a set of proactive secure coding and SDLC-integrated practices that together form a holistic methodology for mitigating vulnerabilities. Unlike narrow attack-focused studies, Khair provides a comprehensive model that integrates security considerations throughout all phases of the SDLC: requirements, design, implementation, testing, deployment, and maintenance.

The following scenarios demonstrate how Khair's concepts can be applied to real-world contexts and highlight how a security-centric SDLC can mitigate vulnerabilities early through secure coding practices, developer education, and continuous security assessment [1].

A. Scenario 1: Financial Payment Gateway

A financial service provider is developing a payment gateway that supports credit card transactions, recurrent payment bills, and integrating merchant APIs. Auditors found a number of vulnerabilities and weaknesses throughout the initial phases in the prototype assessment: Poorly validated transaction inputs, error messages that showed internal SQL structure, and no security requirements on third party components.

1) Threats and Vulnerabilities:

- **SQL Injection Risk:** The financial query strings are created directly by concatenating the user inputs.
- **Disclosure of Information:** SQL syntax and table layout are shown in error messages.
- **Weak Authentication:** Single-factor, which is vulnerable to credential stuffing on the basis of one password.
- **Session Hijacking Risks:** Never changing or regenerating the session ID after login.

2) Relevant Contribution of Research Paper:

Khair observes that under-documented or unknown security requirements, which are discovered too late, contribute most vulnerabilities to the design process and result in insecure design decisions and ad-hoc fixes [1]. Conventional SDLCs are characterized by functionality and deadlines, which tend to push security at the last phases where remediation is expensive. Security-integrated SDLC by Khair emphasizes the significance of establishing clear security goals and implementing secure coding techniques, including input validation, output encoding and safe error handling in all the stages of development.

3) Translation of Research Paper Concepts into Scenario 1:

Using Khair's framework, the team comes up with clear security requirements at the requirements

stage, such as protection against the injection attack, the encryption of finances, and the compliance of the PCI-DSS.

The design phase is to re-architecture the workflow to ensure that all inputs to the transaction are redirected via centralized validating layers, error-handlers eliminate internal explosibility and structure, and merchant API integrations are based on authenticated and signed exchange.

During implementation, the developers implement secure coding, such as sanitization, least privilege, and controlled exception handling.

In testing, the team comes up with security concern test cases according to requirements, conducts code reviews, and relies on automated vulnerability scanning tool as suggested by Khair.

Mitigation (Based on Khair's Framework):

- **Validation/Parameterization of Input:** Use strong validity checks and parameterized queries.
- **Secure Error Handling:** Replace detailed SQL error descriptions with general errors.
- **Upgrade Authentication:** Implement MFA, password complexity, and rate limiting.
- **Session Security Measures:** Rotate session IDs; Maintain secure and HttpOnly cookie flags.
- **SDLC Security Integration:** Prepare security requirements in early stages, conduct reviews focused on security, and integrate SAST/DAST tools [1].

4) Critical Analysis:

The scenario reflects the way in which the model by Khair is able to transform the development culture of reactive vulnerability patching to proactive security-oriented engineering. Yet, Khair points at a weakness: implementation requires a lot of developer training and corporate dedication [1]. Even a well-maintained policy cannot work under strict deadlines or a lack of coding discipline.

B. Scenario 2: Mobile Healthcare Application for Patient Records

A hospital launches a mobile application that allows patients to access medical records, laboratory reports, prescriptions, and secure messages. In a security audit, it is found that there is a lack of role-based access control, the insecure storage of tokens, and system logs showing the identification of the patients and timestamps.

1) Threats and Vulnerabilities:

- **Broken Access Control:** Patient data can be accessed directly in the API without server-side patient role checking.
- **Insecure Data Storage:** Tokens of authentication are kept and stored in plaintext.
- **PHI Exposure in Logs:** Logs contain identifiers and timestamps.
- **Unencrypted (plaintext) Communications:** endpoint(s) communicate in plaintext instead of enforcing TLS.

2) Relevant Contribution of Research Paper:

According to Khair, secure configuration processes, strong authentication/ authorization and secluded data-protection purposes are critical to secure systems [1]. Security vulnerabilities, such as poor access control and unsecured data treatment are caused by the lack of attention to security at the design phase. Khair emphasizes the role of proactive security, especially in fields like healthcare, because the cost of breaches is generally high as data is crucial in the healthcare sector.

3) Translation of Research Paper Concepts into Scenario 2:

Following Khair's methodology, the team redefines requirements to reflect confidentiality rules and access-control conditions for different user roles (patient, clinician, administrator).

In the design phase, PHI protection is introduced in architectural diagrams such as secure token storage strategies, encryption requirements, and log-redaction policies.

During implementation, developers use secure coding practices to encrypt sensitive data both in transit and at rest, implement access control on the server side, and avoid exposure of PHI through logs.

In testing, the team conducts API abuse test, logging test, and role-based access test, which fits the suggestion of Khair that security testing should be incorporated into the process of QA.

4) Mitigation (Based on Khair's Framework):

- **Role-Based Access Control:** Enforce and integrate least privilege with strict server-side RBAC.
- **Secure Token Storage:** Use OS-level keystore mechanisms.
- **Encrypted Data at Rest and Transit:** Apply strong cryptography and require TLS 1.3.
- **Log Sanitization:** Remove PHI traces to follow secure error-handling practices.
- **Security-by-design:** Support early inclusion of privacy requirements and conduct audits frequently.

5) Critical Analysis:

This scenario shows that Khair's integrated SDLC ensures confidentiality through coordinated layers of secure practices [1]. Nevertheless, organizational limitations, such as limited staff resources or inconsistent developer dedication, may obstruct full implementation.

C. Scenario 3: Smart-Home IoT Ecosystem with Multi-Device Connectivity

A smart-home system makes use of a central hub to control cameras, smart locks, thermostats, and sensors. Analysts find insecure firmware update systems

implemented, enabled production debug interfaces, and unsafe memory functions that make buffer-overflow vulnerabilities.

1) Threats and Vulnerabilities:

- **Firmware Hijacking:** The system update protocol does not authenticate using a signature, so it can be altered off-line and put back into service.
- **Buffer Overflow Risks:** The use of tainted memory functions, such as strcpy, corrupts system memory and then is susceptible to hijacking.
- **Debug Interfaces Enabled:** Production devices expose debug interfaces.
- **Default Credentials:** Factory default credentials can be easily misused by attackers.

2) Relevant Contribution of Research Paper:

Khair highlights secure configuration, memory-safe coding, and continuous vulnerability assessment as essential SDLC components [1]. IoT environments, due to constrained resources, require even stricter adherence to secure coding techniques because they lack extensive defensive layers.

3) Mitigation (Based on Khair's Framework):

- **Process of Secure Firmware Update:** Require signed firmware, use mutual TLS, and perform integrity checks to authenticate.
- **Memory-Safe Programming Practices:** Swap out un- safe functions with bounded alternatives that promote memory safety and threat elimination.
- **Secure Configuration Management:** Disable debug interfaces and require compulsory password changes for more secure configurations.
- **SDLC-Integrated Security Testing:** Add SAST to detect memory issues and apply penetration testing to find vulnerabilities.

4) Critical Analysis:

This scenario proves that Khair's framework could reduce firmware and hardware-level threats in IoT ecosystems. However, the model assumes sufficient computational resources to support robust security controls. In resource-constrained IoT devices, full implementation may require compromises or optimization of trade-offs.

IV. Results, Discussion, and Evaluation

The scenario analysis shows how the security-focused SDLC framework of Khair [1] can be implemented in various areas such as financial services, healthcare, and IoT to show effectiveness and draw practical constraints.

A. Results

1) Financial Payment Gateway:

The security features – input sanitation, encryption and adherence to PCI-DSS were discovered and defined early on, so the developers could react in a preventative way with the vulnerabilities. The transaction input was designed under the influence of centralized validation layers, and error messages did not have to open any part of SQL structure or system internals. The instantiation was built according to a secure coding technique, for example parametrized queries, least privilege approach and session management control. Testing was performed using security-related test cases, code review and automated SAST scan. All the above practices minimize the exposure to SQL injection attacks, information leakage and session hijacking, which validate Khair's statement that secure practices are best implemented early in the life cycle of SDLC never to incur high cost of mitigation in later stages of development [1].

2) Mobile Healthcare Application for Patient Records:

The healthcare example clearly illustrates the importance of confidentiality and patient data protection. At the compliance level, role-based access control (RBAC) separated patient, clinician and administrator access. Considerations, which are happenings during design include secure token storage and encryption policy (including logging redaction). To ensure secure coding practice, the developers enforced encryption for at rest and in-transit data, server-side access control and restricted logging. We tested by simulating API abusive patterns, reviewing logs and assessing access control.

3) Smart-Home IoT Ecosystem with Multi-Device Connectivity:

There are other constraints in IoT devices, which include limited processing horsepower and memory weakness. Requirements were, among others, that firmware must be signed, default settings had to be secure and all code writing should happen in a memory-safe manner. There were debug interfaces disabled during design, and security updating mechanisms introduced. The solution included replacing insecure memory operations with secure ones and using secure default credentials and built-in security firmware verification. Tests composed static code analysis and configuration, memory-safe penetration testing. This example shows that the framework of Khair [1] is flexible, showing that adaptation within these domains is necessary when implementing security-oriented practices in confined environments. It is well-supported by research that memory-safe development and configuration management are key contributors to IoT security [3].

B. Discussion

• **Effectiveness Framework:** The model presented by Khair as successful in all three cases and it was applied to sew information security practices in the work of system development life cycle, decreasing vulnerabilities associated risk and enhancing software reliability [1]. You need details about security needs as quickly as you are able to so that defects can be addressed in the beginning before pricey solutions tend to be needed.

• **Human and Organizational:** Factors A developer's training and the organizational commitment are the key for adoption success. Even a good design might not succeed if

people are unaware of it [2]. You can get compliance help from tools like SAST and automated testing; but that piece of non-technical knowledge and urge to stay aware of security issues? You cannot automate that.

• **Domain-Specific Concerns and Tradeoffs:** Each domain brings its challenges. For instance, IoT systems could need to tradeoff between computational resources and the security policies, while financial legacy hardware solutions might be hard to interface. The goals of security need to be balanced with operational requirements by prioritization and risk based decision making [1], [3].

• **Comparative Literature:** Our results are supported by other investigations. Software engineering research on secure software development indicates that the SDLC and its security practices improve maintainability and reduce exploitable vulnerabilities [2],[3]. Additionally, secure configuration, access control and memory safe programming is also frequently cited as addressable levers to further reduce risk in domain-specific settings.

C. *Evaluation*

1) Strenghts:

- Scenario-based approach illustrates the practical use of Khair's framework across domains.
- The model is actionable, with it highlighting trade-offs, organizational considerations, and adaptation specific to domains.
- Emphasizes cost-effectiveness of proactive security integration.

2) Limitations:

- Analysis is conceptual, there no quantitative measures of vulnerability reduction or cost savings were gathered.
- The implementation assumes availability of automated tools, trained developers, and organizational support.
- Generalizability is limited; cloud-native architecture, large-scale regulatory environment, and real-time systems are not covered in the three scenarios.

V. Conclusion and Future Work

This paper confirms that the integration of secure coding practices across all SDLC, phases as per Khair [1], greatly improves the security, maintainability and stability of software. Findings on scenarios in financial, healthcare and IoT realms revealed pro-active threat mitigation, secure requirement specification, secure design, code discipline and integrated testing play a major role in reducing the attack surface area.

References

[1] M. A. Khair, "Security-Centric Software Development: Integrating Secure Coding Practices into the Software Development Lifecycle," *Technology & Management Review*, vol. 3, no. 1, pp. 12–26, 2018.

[2] A. A. Osazee and I. D. Nomaren, "Secure Software Engineering: A Synthesis of SSDLC, DevSecOps, and AI-Driven Threat Mitigation," *Int. J. Dev. Math.*, 2025.

[3] M. Fauzi, V. R. Mohan, Y. Qi, C. Chandrasegar, and S. Muzafar, "Secure Software Development: Best Practices," *Int. J. Emerg. Multidisciplinaries: Comp. Sci. & AI*, 2023.

[4] R. A. Khan, S. U. Khan, H. U. Khan, and M. Ilyas, "Systematic Literature Review on Security Risks and Its Practices in Secure Software Development," IEEE Access, vol. 10, pp. 5456–5481, 2022.

[5] I. Ryan, U. Roedig, and K.-J. Stol, "Measuring Secure Coding Practice and Culture: A Finger Pointing at the Moon Is Not the Moon," in Proc. 45th IEEE/ACM Int. Conf. on Software Engineering (ICSE), 2023, pp. 1622–1634.